Aula 3 - Login e Perfis de Usuário

Objetivos da aula

- Criar um endpoint de Login na API Java;
- Buscar usuários no banco pela matrícula;
- Validar matrícula e senha;
- Retornar os dados do usuário autenticado;
- Consumir um endpoint POST no Flutter;
- Criar uma tela de Login;
- Controlar navegação baseada em autenticação;
- Diferenciar usuários Administradores e Funcionários.

Problema proposto

Na empresa existem dois tipos de usuários:

Funcionário

Pode:

- Consultar EPIs;
- Solicitar empréstimo.

Administrador

Pode:

- Consultar EPIs;
- Aprovar empréstimos;
- Registrar devoluções.

Parte 1 - Preparando o banco de dados

a) Adicionar coluna perfil no gerenciador do Banco de Dados:

ALTER TABLE funcionario ADD COLUMN perfil VARCHAR(20);

b) Atualizar os registros existentes

UPDATE funcionário SET perfil='ADMIN' WHERE matricula='1001';
UPDATE funcionário SET perfil='FUNCIONARIO' WHERE matricula='1002';
UPDATE funcionário SET perfil='FUNCIONARIO' WHERE matricula='1003';

Obs1: Estou usando os dados que foram inseridos na aula1 na tabela funcionário. Caso dê erro, é porque não foram encontradas as matrículas que deveriam ter sido inseridas, então veja o conteúdo da sua tabela funcionario
Obs2: Fique a vontade caso queira inserir dados de outros funcionários na sua base de dados

c) Verificando: SELECT * FROM funcionario;
Resultado esperado:
1001 Ana Souza ADMIN
1002 Bruno Lima FUNCIONARIO
1003 Carla Mendes FUNCIONARIO

Parte 2 - Atualizando a Entidade
Arquivo: Funcionario.java
Adicionar:
@Column(nullable = false, length = 20)
private String perfil;

Etapa 3 – Atualizar o Repository
Arquivo: FuncionarioRepository

Adicionar:
Optional<Funcionario> findByMatriculaAndSenha(
String matricula,
String senha);

Explicação - O Spring criará automaticamente uma consulta semelhante a:
SELECT * FROM funcionário WHERE matricula = ? AND senha = ?;

Etapa 4 – Criar AuthController

Criar: AuthController

Código:

```java
package com.senac.api_epi.controllers;

import com.senac.api_epi.entities.Funcionario;
import com.senac.api_epi.repositories.FuncionarioRepository;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
@RestController
@RequestMapping("/auth")
public class AuthController {
    private final FuncionarioRepository repository;  2 usages
    public AuthController(
            FuncionarioRepository repository) {
        this.repository = repository;
    }
    @PostMapping("/login")
    public ResponseEntity<?> login(
            @RequestBody Funcionario funcionarioLogin) {
        Funcionario funcionario =
                repository.findByMatriculaAndSenha(
                        funcionarioLogin.getMatricula(),
                        funcionarioLogin.getSenha())
                        .orElse( other: null);
        if (funcionario == null) {
            return ResponseEntity
                    .badRequest()
                    .body("Matrícula ou senha inválidos");
        }
        return ResponseEntity.ok(funcionario);
    }
}
```

Explicação - Este endpoint: POST /auth/login

recebe matrícula e senha.
Se o usuário existir: devolve os dados do funcionário.
Se não existir: devolve mensagem de erro.

@RequestBody Funcionario funcionarioLogin
Recebe os dados enviados pelo Flutter:

{
"matricula": "1001",
"senha": "1234"
}
e os transforma em um objeto Java.
Por isso podemos acessar:
funcionarioLogin.getMatricula()
funcionarioLogin.getSenha()
para realizar a consulta no banco de dados.

Fluxo do Login
Flutter
↓
POST /auth/login
↓
Funcionario funcionarioLogin
↓
Repository
↓
Banco de Dados
↓
Usuário encontrado?
↓
Sim → retorna os dados do funcionário
Não → retorna erro
Etapa 5 – Testando no Postman

URL
POST
http://localhost:8080/auth/login

Body
{
"matricula": "1001",
"senha": "1234"
}

Resposta esperada
{
"id": 1,
"matricula": "1001",
"nome": "Ana Souza",
* "setor": "Manutenção",

*"perfil": "ADMIN"
}

Observe que: “senha” não aparece
O motivo é a cláusula que inserimos na entidade: @JsonProperty(access = JsonPro*erty.Access.WRITE_ONLY)

Teste uma matricula ou senha inválida para averiguar a mensagem

Etapa 6 – Criando o Model Usuario no Flutter
Arquivo: models/usuario.dart

usuario.dart

lib > src > models > usuario.dart > ...

Windsurf: Refactor | Explain

```dart
class Usuario {
  final int id;
  final String matricula;
  final String nome;
  final String setor;
  final String perfil;
  Usuario({
    required this.id,
    required this.matricula,
    required this.nome,
    required this.setor,
    required this.perfil,
  });
  factory Usuario.fromJson(
      Map<String, dynamic> json) {
    return Usuario(
      id: json['id'],
      matricula: json['matricula'],
      nome: json['nome'],
      setor: json['setor'],
      perfil: json['perfil'],
    );
  }
}

```

O que faz o método fromJson()?
Ele converte os dados recebidos da API em um objeto Dart.

Exemplo:
JSON recebido:
{
"id": 1,
"matricula": "1001",
"nome": "Ana Souza",
"setor": "Manutenção",
"perfil": "ADMIN"
}

↓
Objeto:
Usuario

Etapa 7 – Criando o AuthService
Arquivo: services/auth_service.dart




```dart
import 'dart:convert';
import 'package:http/http.dart' as http;
import '../models/usuario.dart';
import 'api_config.dart';

class AuthService {
  Future<Usuario> login(
      String matricula,
      String senha) async {
    final response =
        await http.post(
      Uri.parse(
          '${ApiConfig.baseUrl}/auth/login'),
      headers: {
        'Content-Type': 'application/json'
      },
      body: jsonEncode({
        'matricula': matricula,
        'senha': senha,
      }),
    );
    if (response.statusCode == 200) {
      return Usuario.fromJson(
        jsonDecode(response.body),
      );
    }
    throw Exception(
      'Matrícula ou senha inválidos',
    );
  }
}
```

O que esta classe faz?
Ela é responsável por:
enviar os dados de login para a API;
receber a resposta;
converter o JSON em um objeto Usuario.

Etapa 8 – Criando a LoginScreen
Arquivo: screens/login_screen.dart



```
34              usuario: usuario,
35            ), // MainScreen
36          ), // MaterialPageRoute
37        );
38      } catch(e) {
39        ScaffoldMessenger.of(context)
40            .showSnackBar(
41          SnackBar(
42            content: Text(e.toString()),
43          ), // SnackBar
44        );
45      } finally {
46        setState(() {
47          carregando = false;
48        });
49      }
50    }
51    @override
      Windsurf: Refactor | Explain | Generate Function Comment | ×
52    Widget build(BuildContext context) {
53      return Scaffold(
54        appBar: AppBar(
55          title:
56              const Text('Login'),
57          backgroundColor:
58              Colors.blue,
59          foregroundColor:
60              Colors.white,
61        ), // AppBar
62        body: Padding(
63          padding:
64              const EdgeInsets.all(20),
65          child: Column(
66            children: [
```

Esta linha 34 terá este erro: usuário: usuário que só será resolvido quando a tela main_screen.dart for atualizada, passando os dados do usuário para esta tela.

```
67                const SizedBox(height: 30),
68                TextField(
69                  controller:
70                      matriculaController,
71                  decoration:
72                      const InputDecoration(
73                    labelText: 'Matrícula',
74                    border:
75                        OutlineInputBorder(),
76                  ), // InputDecoration
77                ), // TextField
78                const SizedBox(height: 20),
79                TextField(
80                  controller:
81                      senhaController,
82                  obscureText: true,
83                  decoration:
84                      const InputDecoration(
85                    labelText: 'Senha',
86                    border:
87                        OutlineInputBorder(),
88                  ), // InputDecoration
89                ), // TextField
90                const SizedBox(height: 30),
91                SizedBox(
92                  width: double.infinity,
93                  child: ElevatedButton(
94                    onPressed:
95                        carregando
96                            ? null
97                            : fazerLogin,
98                    child: carregando
99                        ? const CircularProgressIndicator()
100                       : const Text(
```

```
101                           'Entrar',
102                       ), // Text
103                   ), // ElevatedButton
104               ), // SizedBox
105             ],
106           ), // Column
107         ), // Padding
108       ); // Scaffold
109     }
110   }
```

Explicando o código

Controllers

final TextEditingController matriculaController = TextEditingController(); // Controla o campo Matrícula.
final TextEditingController senhaController = TextEditingController(); // Controla o campo Senha.

Método fazerLogin()

Future<void> fazerLogin()
Responsável por:

1. Ler os campos;
2. Chamar a API;
3. Receber os dados do usuário;
4. Abrir a tela principal.

Indicador de carregamento

carregando = true; // Desabilita o botão e exibe:
CircularProgressIndicator() // enquanto a API responde.

Tratando erros

catch(e) // Caso o login falhe:
Matrícula ou senha inválidos
será exibida uma mensagem na tela.

Etapa 9 – Ajustando a MainScreen

lib > src > screens > main_screen.dart > MainScreen > build

```dart
import 'package:flutter/material.dart';

import 'epi_screen.dart';
import 'funcionario_screen.dart';

import '../models/usuario.dart'; // adicionado para importar a classe Usuario
import 'login_screen.dart'; // para sair

Windsurf: Refactor | Explain
class MainScreen extends StatelessWidget {
  final Usuario usuario; // adicionado para armazenar o usuário logado
  const MainScreen({
    super.key,
    required this.usuario,
  }); // adicionado para receber o usuário logado

  @override
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Empréstimos de EPIs'),
        backgroundColor: Colors.blue,
        foregroundColor: Colors.white,
      ), // AppBar
```

Veja as linhas alteradas com a informação comentada para realizar no seu arquivo:

```
      drawer: Drawer(
        child: ListView(
          children: [
            UserAccountsDrawerHeader(
              // adicionado para exibir informações do usuário logado
              accountName: Text(usuario.nome), // exibe o nome do usuário logado
              accountEmail: Text(
                usuario.perfil,
              ), // exibe a matrícula do usuário logado // Text
              currentAccountPicture: CircleAvatar(
                backgroundColor: Colors.white,
                child: Text(
                  usuario.nome[0], // exibe a primeira letra do nome do usuário logado
                  style: const TextStyle(fontSize: 40.0),
                ), // Text
              ), // CircleAvatar
            ), // UserAccountsDrawerHeader
            // const DrawerHeader(
            //   decoration: BoxDecoration(
            //     color: Colors.blue,
            //   ),
            //   child: Text(
            //     'Menu Principal',
            //     style: TextStyle(
            //       color: Colors.white,
            //       fontSize: 22,
            //     ),
            //   ),
            // ),

            ListTile(
              leading: const Icon(Icons.home),
              title: const Text('Início'),
              onTap: () {
```

```
        body: Center(
          child: Text(
            'Bem-vindo ${usuario.nome}',
            style: const TextStyle(fontSize: 18),
          ), // Text
          // child: Text(
          //   'Selecione uma opção no menu lateral',
          //   style: TextStyle(fontSize: 18),
          // ),
        ), // Center
      ); // Scaffold
```

Etapa 10 – Alterando o main.dart

```dart
lib > main.dart > ...
import 'package:app_epi/src/screens/login_screen.dart';
import 'package:flutter/material.dart';

//import 'src/screens/main_screen.dart';

Run | Debug | Profile | Windsurf: Refactor | Explain | Generate Function Comment | ×
void main() {
  runApp(const MyApp());
}

Windsurf: Refactor | Explain
class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Windsurf: Refactor | Explain | Generate Function Comment | ×
  Widget build(BuildContext context) {
    return MaterialApp(
      debugShowCheckedModeBanner: false,
      title: 'Controle de EPIs',
      home: LoginScreen(), //const MainScreen(),
    ); // MaterialApp
  }
}
```

Rode sua aplicação
A tela de login será a primeira tela, e após a confirmação você passa a ter acesso a todo o sistema
Experimente o usuário administrador e depois outro usuário comum~

Para finalizar, vamos inserir a opção de sair

Etapa 11 – Sair no menu

Em main_screen.dart, onde fica nosso menu, você já fez no início a importação da tela que será chamada:

import 'login_screen.dart'; *// para sair*

Agora acrescente abaixo do seu último ListTile a opção abaixo:

```dart
// inserir a opção de sair do sistema no menu lateral
ListTile(
  leading: const Icon(Icons.logout, color: Colors.red),
  title: const Text('Sair'),
  onTap: () {
    Navigator.pushAndRemoveUntil(
      context,
      MaterialPageRoute(builder: (_) => const LoginScreen()),
      (route) => false,
    );
  },
), // ListTile
```

Desafio para o término:

Exibir mensagens diferentes para cada perfil:

ADMIN

Bem-vindo nome

Você possui permissões para aprovar empréstimos e registrar devoluções

FUNCIONARIO

Bem-vindo nome

Você pode consultar EPIs e solicitar empréstimos.